

SQL Practice Questions 26-50

Intermediate SQL workbook for SQLite



WHAT YOU WILL PRACTISE

- Advanced aggregation and HAVING
- Self joins and manager comparisons
- CASE expressions and salary analysis
- RANK, DENSE_RANK, LAG and running totals
- Customer summaries with LEFT JOIN and COALESCE

Question-only edition | Based on employees, customers, products and orders tables

How to use this workbook

Solve each challenge independently, test it against your SQLite dataset, and explain why your grouping, join or window function produces the intended result.

1. Understand	Identify the output grain and required tables.
2. Build	Write the simplest correct query first.
3. Test	Check NULLs, duplicate values and ties.
4. Refine	Improve aliases, ordering and readability.

Workbook map

Level	Focus	Questions
6	Employee Aggregation	Q26-Q35
7	Employee Comparisons	Q36-Q45
8	Customer Order Analysis	Q46-Q50

LEVEL 6 Employee Aggregation

Practise grouped calculations, filtering of groups and manager reporting structures.

Q26

Department headcount

Find the number of employees in each department.

Expected: department | employee_count

GROUP BY / COUNT

Q27

Department salary expenditure

Find the total salary paid to employees in each department.

Expected: department | total_salary

GROUP BY / SUM

Q28

Large departments

Find departments that have at least four employees.

Filter grouped results after counting employees.

HAVING

Q29

Employees without managers

Find employees whose manager_id is NULL.

Return the employee ID, name, department and salary.

NULL FILTERING

Q30

Direct-report count

Find the number of employees reporting directly to each manager.

Expected: manager_name | direct_reports

SELF JOIN / COUNT

Q31

Managers with multiple reports

Find managers who supervise at least three employees.

Return only qualifying managers and their direct-report count.

SELF JOIN / HAVING

Q32

Salary range by department

Find the minimum and maximum salary in each department.

Expected: department | minimum_salary | maximum_salary

MIN / MAX

Q33

Department salary range difference

Calculate the difference between the highest and lowest salary in each department.

Expected: department | salary_range

AGGREGATION

Q34

Department salary summary

Display employee count, total salary and average salary for each department.

Expected: department | employee_count | total_salary | average_salary

MULTIPLE AGGREGATES

Q35**Departments above company average**

Find departments whose average salary is greater than the overall company average salary.

Compare a grouped average with a scalar subquery.

SUBQUERY / HAVING

LEVEL 7 Employee Comparisons

Apply self joins, CASE expressions and window functions to employee salary analysis.

Q36**Employee and manager details**

Display each employee together with their manager's name and salary.

Expected: employee | employee_salary | manager | manager_salary

SELF JOIN

Q37**Salary difference from manager**

Calculate the salary difference between each employee and their manager.

Expected: employee | manager | salary_difference

SELF JOIN / CALCULATION

Q38**Employees earning at least 90% of manager salary**

Find employees whose salary is at least 90% of their manager's salary.

Use the employee-manager relationship for the comparison.

SELF JOIN / FILTER

Q39**Salary classification**

Classify employees as High, Medium or Low using CASE.

High: >= 75,000; Medium: 55,000 to < 75,000; Low: < 55,000.

CASE

Q40**Department salary rank**

Rank employees by salary within their respective departments.

Show employee, department, salary and salary rank.

RANK / PARTITION BY

Q41**Bottom-paid employee by department**

Find the lowest-paid employee in each department.

Return all employees tied at the lowest salary.

DENSE_RANK

Q42**Top three salaries by department**

Find the three highest distinct salaries in each department.

Use a ranking function that handles duplicate salaries correctly.

DENSE_RANK

Q43**Employees tied for highest salary**

Find departments where multiple employees share the highest salary.
Return the department, employees and shared highest salary.

WINDOW FUNCTIONS**Q44****Salary compared with previous employee**

Use LAG() to compare each employee's salary with the previous salary in the same department.
Order salaries consistently within each department.

LAG / PARTITION BY**Q45****Running departmental salary total**

Calculate a running total of salaries within each department, ordered from highest to lowest.
Show each employee and the cumulative departmental salary.

SUM OVER**LEVEL 8**

Customer Order Analysis

Build customer-level summaries while accounting for customers with no matching orders.

Q46**Customer order summary**

Display total orders and total spending for each customer, including customers with no orders.
Expected: customer | total_orders | total_spending

LEFT JOIN / COALESCE**Q47****Customer order status**

Classify customers as Frequent, Occasional or No Orders based on their order count.
Frequent: >= 3; Occasional: 1-2; No Orders: 0.

CTE / CASE**Q48****Customers with several orders**

Find customers who placed at least three orders.
Return customer ID, customer name and order count.

GROUP BY / HAVING**Q49****Customers with one order**

Find customers who placed exactly one order.
Return customer ID, customer name and order count.

GROUP BY / HAVING**Q50****Customer average order value**

Calculate the average order amount for each customer.
Expected: customer | average_order_value

JOIN / AVG

Completion checklist

- The selected JOIN preserves the required rows.
- Every non-aggregated selected column is grouped correctly.

- HAVING is used for grouped filters and WHERE for row filters.
- Window functions use the correct PARTITION BY and ORDER BY.
- The query handles NULL values and ties deliberately.